

ITCS5010 GenAI:ForgePilot

CLI Coding Assistant — Final Report

Varad Paradkar Nidhi Shah Brinda Chinnaraji
Sai Kiran Jagini Shar Adhiambo

March 27, 2026

Contents

1	Introduction	2
2	Design Goals and Scope	2
2.1	Primary Goals	2
2.2	Scope Boundaries	2
3	System Architecture Overview	2
3.1	Component Responsibilities	3
4	State Diagram	3
5	Retrieval and MCP Pipeline	3
5.1	RAG Pipeline Summary	3
5.2	MCP Server Roles	3
6	Sequence Diagram — Scenario 1 (Documentation Query with RAG)	4
7	Sequence Diagram — Scenario 2 (Read/Edit File Flow)	5
8	Execution Semantics	5
8.1	Agentic Loop	5
8.2	Tool Runtime and Safety Modes	5
8.3	Provider Abstraction and Fallback	6
9	Validation and Testing	6
10	Limitations and Future Work	6
11	Conclusion	6
A	Artifact Index	6

1 Introduction

ForgePilot is a modular CLI coding assistant that combines an agentic loop, provider abstraction, MCP tool orchestration, and a local RAG pipeline. The system supports practical repository workflows such as file inspection, edits, shell execution, and grounded responses using local vector retrieval.

This report presents the architecture, runtime diagrams, implementation details, validation evidence, and limitations/future work.

Repository links:

- **Project Repository:** <https://github.com/vradcar/GenAI-Group-Project-2>
- **Local report source:** docs/FINAL_REPORT.md

2 Design Goals and Scope

2.1 Primary Goals

- Provide an interactive CLI assistant for coding tasks (single-run and REPL).
- Support safe execution via explicit user confirmation mode and fast execution via auto mode.
- Integrate external and local tools through MCP with a consistent runtime interface.
- Ground responses with local document retrieval using a persistent vector store.

2.2 Scope Boundaries

- In-scope: CLI workflows, local tooling, MCP integration, and local RAG retrieval.
- Out-of-scope: multi-tenant cloud architecture, custom model training, and full enterprise auth.

3 System Architecture Overview

The system follows a layered architecture with a user-facing CLI, a core agent/runtime layer, and tool/data backends.

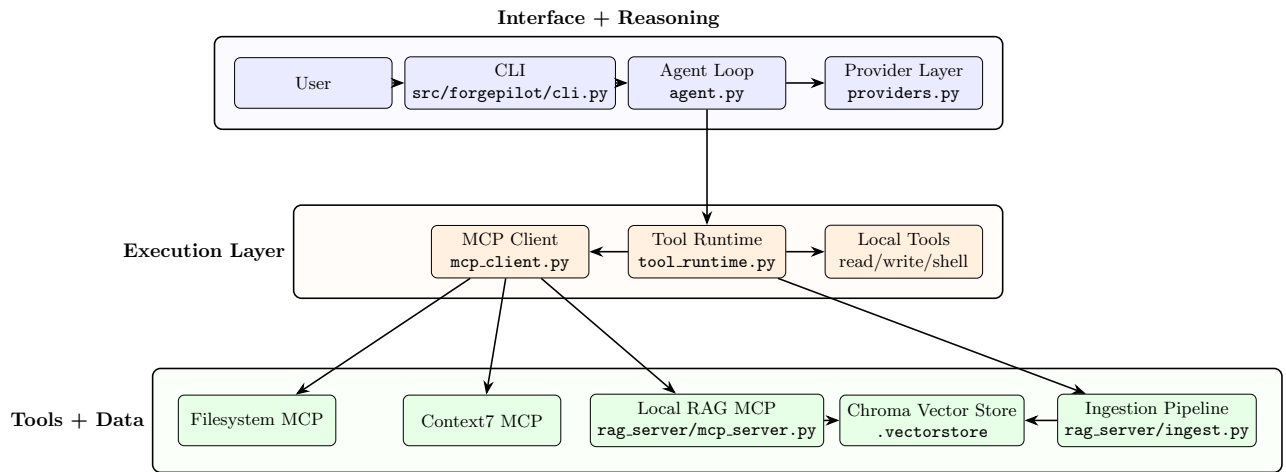


Figure 1: ForgePilot runtime architecture using CLI, agent loop, provider abstraction, MCP, and local RAG.

3.1 Component Responsibilities

Component	Responsibility
CLI (<code>cli.py</code>)	Accept user tasks, run single-task mode or REPL mode, display tool call status and outputs.
Agent Loop (<code>agent.py</code>)	Generate one-step plans, parse structured responses, dispatch actions, and iterate to completion.
Provider Layer (<code>providers.py</code>)	Create model-client abstraction across providers and apply fallback behavior when provider setup fails.
Tool Runtime (<code>tool_runtime.py</code>)	Enforce <code>confirm</code> / <code>auto</code> mode and route calls to local tools or MCP tools.
MCP Client (<code>mcp_client.py</code>)	Connect to MCP servers, discover tools, namespace tool names, and normalize responses.
RAG Modules (<code>rag_server/*</code>)	Ingest docs, create embeddings, retrieve top-k grounded chunks, and expose retrieval via MCP tools.

Table 1: High-level component responsibility map.

4 State Diagram

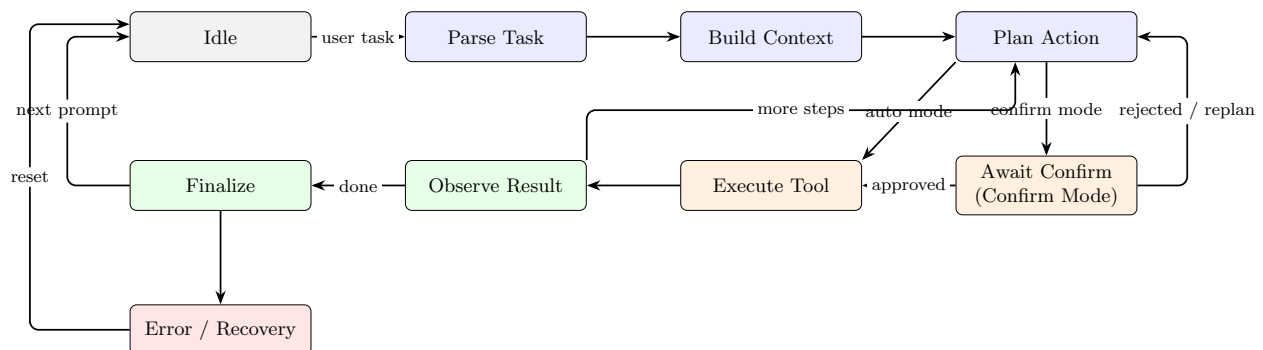


Figure 2: Agent execution lifecycle with confirm/auto branches and recovery path.

5 Retrieval and MCP Pipeline

5.1 RAG Pipeline Summary

- Source docs from `docs/source_docs` are chunked and embedded by `rag_server/ingest.py`.
- Embeddings are persisted in Chroma under `.vectorstore`.
- `rag_server/retriever.py` expands queries and performs fusion-style ranking.
- `rag_server/mcp_server.py` exposes `rag_query_docs` and `rag_health`.

5.2 MCP Server Roles

Server	Type	Purpose
filesystem	MCP (Node)	File and directory operations in workspace scope.
context7	MCP (remote)	External documentation/context retrieval for coding tasks.
rag	MCP (local Python)	Grounded retrieval from local vector store and health checks.

Table 2: Configured MCP servers and their roles.

6 Sequence Diagram — Scenario 1 (Documentation Query with RAG)

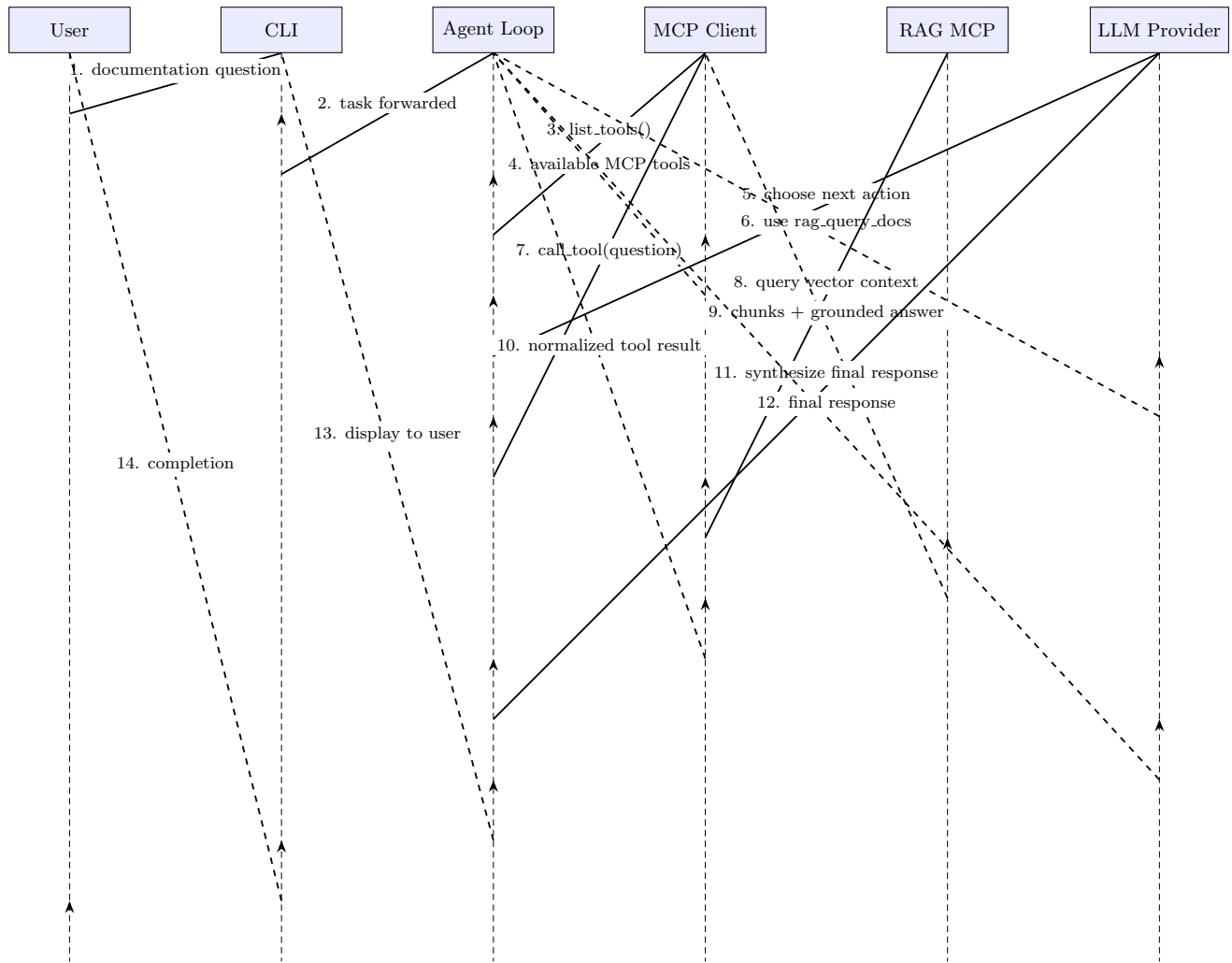


Figure 3: Sequence flow for documentation query grounded with local RAG.

7 Sequence Diagram — Scenario 2 (Read/Edit File Flow)

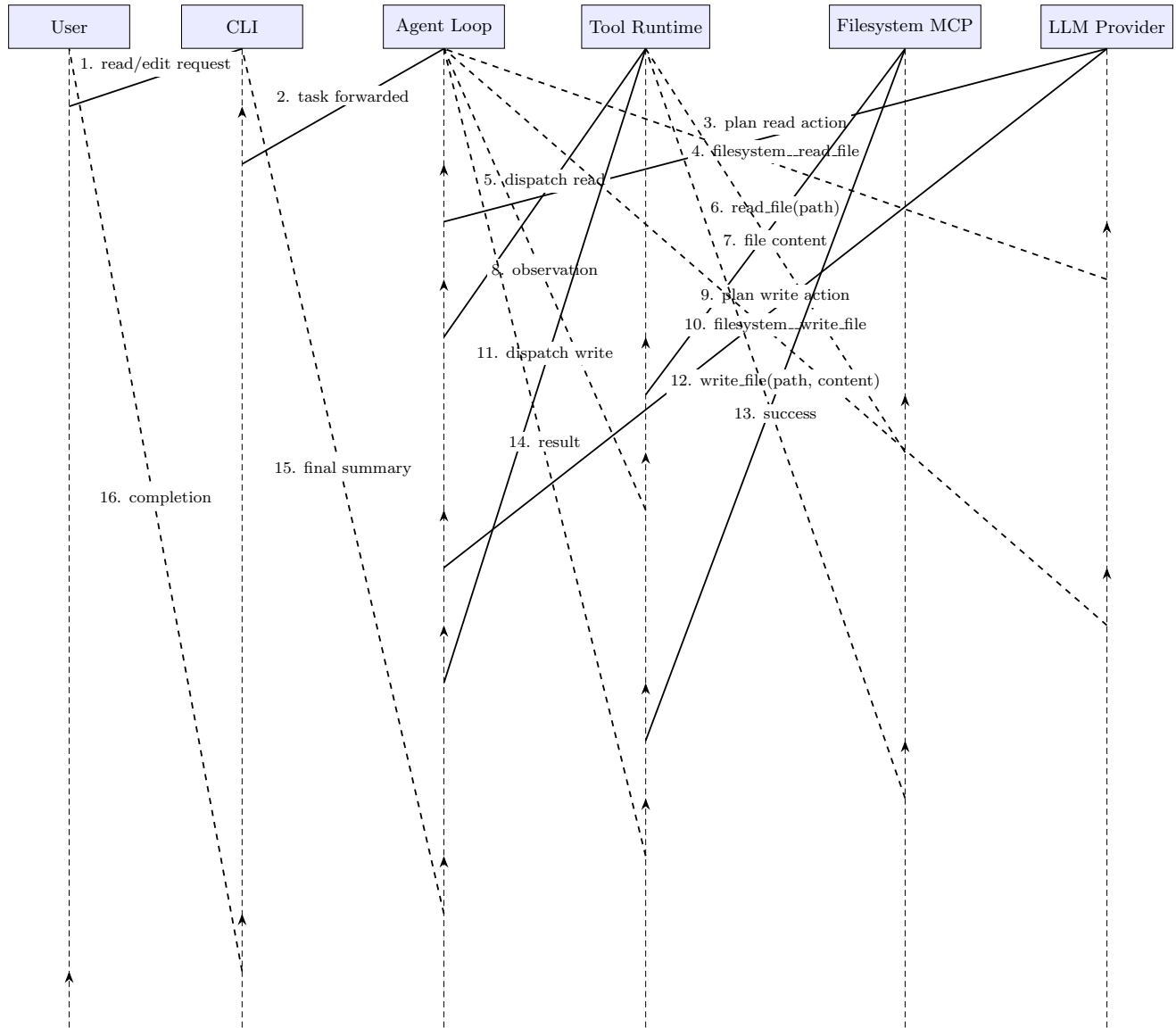


Figure 4: Sequence flow for read/edit workflow through Tool Runtime and filesystem MCP.

8 Execution Semantics

8.1 Agentic Loop

The agent runs in bounded steps (`max_steps`) and exchanges a strict structured format: `thought`, `action`, and `final`. If an action is returned, the call is routed through Tool Runtime; otherwise, completion or replanning occurs.

8.2 Tool Runtime and Safety Modes

- **Confirm mode:** each tool call requires user approval before execution.
- **Auto mode:** tool calls execute directly for faster workflows.

Both local and MCP tool calls share one dispatch path, ensuring consistent logging and control.

8.3 Provider Abstraction and Fallback

Provider selection is based on environment settings. If a selected provider is unavailable, fallback selection is attempted so the CLI remains operational. Runtime provider failures are captured and surfaced as non-crashing error text.

9 Validation and Testing

- End-to-end usage validated through CLI run mode and REPL mode.
- MCP integrations validated across filesystem, context7, and local rag server paths.
- Deployment documentation and checklist maintained in `DEPLOYMENT.md`.
- Usage and recovery paths documented in `HOW_TO_USE.md`.

Area	Status	Evidence
Automated tests	Pass	<code>pytest -q</code> run in project environment
Deployment flow	Documented	<code>DEPLOYMENT.md</code> with checks and post-deploy checklist
User workflow docs	Updated	<code>HOW_TO_USE.md</code> with working REPL/run commands
Demo artifact	Present	<code>docs/demo/demo-video.mp4</code>

Table 3: Validation status summary.

10 Limitations and Future Work

- Provider/model quality depends on quota, capacity, and API behavior.
- Some hosted models may produce incompatible tool-calling outputs.
- Next step: add stricter markdown quality rubric + validation pass in agent loop.
- Next step: extend benchmark suite for retrieval quality metrics across larger corpora.
- Next step: add deterministic schema validation before finalizing generated documentation.

11 Conclusion

ForgePilot delivers a practical and extensible CLI assistant architecture by combining agentic planning, safe runtime execution, MCP tool orchestration, and local RAG grounding. The implemented design balances usability (run/repl workflows), safety (confirm mode), and automation (auto mode) while preserving traceability through logs, docs, and structured tool calls.

The current implementation is deployment-ready for coursework/demo scenarios and provides a clear baseline for future improvements in retrieval evaluation, prompt quality controls, and provider interoperability.

A Artifact Index

- Architecture narrative: `ARCHITECTURE.md`
- State diagram source: `docs/planning/STATE_DIAGRAM.mmd`
- Sequence source 1: `docs/planning/SEQUENCE_SCENARIO_1_DOC_RAG.mmd`
- Sequence source 2: `docs/planning/SEQUENCE_SCENARIO_2_READ_EDIT.mmd`
- Deployment guide: `DEPLOYMENT.md`

- Usage guide: `HOW_TO_USE.md`